Schritt 44 — Production Checklist & Deployment konkret

Dieser Schritt fügt nicht viele neue GraphQL-Features hinzu. Ziel ist es, das aktuelle Projekt in ein System zu verwandeln, das sicher deployt, geprüft, zurückgerollt und überwacht werden kann.

Wir folgen diesem Flow:

```
Development
    ↓
RSpec / Security checks
    ↓
Staging
    ↓
Database migration
    ↓
Deploy Backend
    ↓
Sync Trusted Operations
    ↓
Smoke Test
    ↓
Deploy Frontend
    ↓
Monitoring
    ↓
Rollback n□u có l□i
```

1. Aktuelle Production-Struktur prüfen

Nach 43 Schritten sollte das Projekt ungefähr so aussehen:

```
app/
├── controllers/
│   └── graphql_controller.rb
│
├── graphql/
│   ├── mutations/
│   ├── resolvers/
│   ├── types/
│   ├── sources/
│   └── your_app_schema.rb
│
├── models/
│   ├── user.rb
│   ├── project.rb
│   ├── project_member.rb
│   └── task.rb
│
├── policies/
│   ├── project_policy.rb
│   └── task_policy.rb
│
└── services/
    ├── graphql/
    │   └── request_logger.rb
    ├── projects/
    └── tasks/

config/
├── environments/
│   └── production.rb
├── initializers/
│   └── rack_attack.rb
└── routes.rb

spec/
├── graphql/
├── policies/
├── services/
└── requests/
```

Wenn Business-Logik noch in großem Umfang in QueryType oder MutationType liegt, sollte sie vor Produktion refactort werden.

2. Production Environment

Prüfen: config/environments/production.rb

Rails Production muss mit RAILS_ENV=production laufen.

Prüfen mit:

```
RAILS_ENV=production bin/rails runner 'puts Rails.env'

# K□t qu□:
production
```

3. Environment Variables

Production benötigt mindestens:

```
DATABASE_URL
SECRET_KEY_BASE
REDIS_URL
RAILS_ENV
```

Je nach Authentication/Deployment zusätzlich:

```
FRONTEND_URL
SENTRY_DSN
SMTP_*
GRAPHQL_PRO_LICENSE
```

wenn das Projekt sie tatsächlich verwendet.

DATABASE_PASSWORD = "mypassword" nicht im Source Code schreiben.

Nicht committen:

```
.env.production
master.key
credentials key
database password
Redis password
API token
```

4. Secrets vor Deployment prüfen

Prüfen mit:

```
bin/rails credentials:show
```

aber die Ausgabe niemals in Git, CI-Logs, Slack oder Tickets kopieren.

In Production sollte die Anwendung Secrets nur über die Umgebung oder einen Secret Manager der Plattform erhalten.

5. Production-Datenbank

Beispiel PostgreSQL:

```
Rails
  ↓
DATABASE_URL
  ↓
PostgreSQL
```

Connection prüfen:

```
RAILS_ENV=production bin/rails runner 'puts ActiveRecord::Base.connection.database_version'
```

Wenn es hier einen Fehler gibt, muss man noch nicht an GraphQL denken — die Datenbankverbindung funktioniert noch nicht.

6. Connection Pool

Angenommen, der Server hat:

```
2 Puma workers
×
5 threads
```

dann können allein für den Web-Prozess ungefähr 10 Datenbankverbindungen nötig sein.

config/database.yml kann verwenden:

```
production:
  url: <%= ENV["DATABASE_URL"] %>
  pool: <%= ENV.fetch("RAILS_MAX_THREADS", 5) %>
```

Bei mehreren Workers, Sidekiq/Background Jobs, Console usw. muss die Gesamtzahl der Connections berechnet werden.

Nicht: Puma concurrency = 50 / DB max connections = 20 und erwarten, dass das System stabil bleibt.

7. Database Constraints

Nicht nur auf Rails Validation verlassen.

Beispiel ProjectMember:

```
validates :user_id,
  uniqueness: {
    scope: :project_id
  }
```

Die Datenbank braucht ebenfalls einen Unique Index:

```
add_index :project_members,
```

```
          [:project_id, :user_id],
          unique: true
```

Production hat parallele Requests. Zwei Requests können gleichzeitig die Model Validation passieren, bevor einer committet. Der Database Constraint ist die letzte Schutzschicht.

8. Indexes prüfen

Das aktuelle Projekt sollte reale Queries prüfen wie:

```
Task.where(
  project_id: project.id,
  status: "todo"
)
```

Möglicherweise nötig:

```
add_index :tasks,
          [:project_id, :status]
```

Assignment:

```
Task.where(
  assignee_id: user.id,
  status: "todo"
)
```

kann benötigen:

```
add_index :tasks,
          [:assignee_id, :status]
```

Membership:

```
add_index :project_members,
          [:project_id, :user_id],
          unique: true
```

Aber nicht jede Column indexieren. relation.explain verwenden, um den Query Plan zu prüfen.

9. Production-Migrationen müssen sicher sein

Vor dem Deployment:

```
bin/rails db:migrate
```

Bei Production-Migrationen nicht nur denken: lokal läuft die Migration → Production ist auch OK.

Beispiel für eine große Tabelle:

```
add_column :tasks,
           :status,
           :string,
           null: false,
           default: "todo"
```

Je nach Datenbank/Version/Tabellengröße kann eine große Änderung Locks verursachen oder das Deployment beeinflussen.

Besser in mehreren Schritten denken:

```
add nullable column
↓
deploy
↓
backfill
↓
validate data
↓
add constraint
```

10. Columns nicht zu früh rename/drop

In Schritt 43 ging es um Schema Evolution.

Beispiel: projects.name → projects.title.

Nicht in einem Deployment:

```
rename_column :projects,
              :name,
              :title
```

während alter Code noch läuft.

Sicherer:

```
Deploy A
→ thêm title

Deploy B
→ dual read/write hoặc migrate
```

```
Deploy C
→ frontend dùng title

Deploy D
→ remove name kh□i GraphQL

Deploy E
→ remove DB column cũ
```

11. Production Authentication

GraphqlController:

```
context = {
  current_user: current_user
}
```

Das schützt GraphQL aber nicht automatisch.

Resolver/Mutation müssen weiterhin:

```
authenticate_user!
```

Beispiel:

```
def resolve
  user = authenticate_user!

  # ...
end
```

12. Production Authorization

Mutation:

```
def resolve(id:, attributes:)
  authenticate_user!

  project =
    Project.find(id)

  authorize!(
    project,
    :update?
  )

  # ...
end
```

Collection:

```
ProjectPolicy::Scope
  .new(current_user, Project)
  .resolve
```

Zu prüfen: Single record → Policy; Collection → Policy::Scope.

Nicht nur Admin testen. Testen: admin, owner, manager, developer, viewer, stranger.

13. Permission Matrix ausführen

Vor Production:

```
bundle exec rspec spec/policies
```

und die Permission-Matrix-Specs.

Beispiel:

```
                admin owner manager dev viewer stranger

View Project      ✓     ✓     ✓      ✓    ✓      ✗
Update Project    ✓     ✓     ✓      ✗    ✗      ✗
Delete Project    ✓     ✓     ✗      ✗    ✗      ✗
Manage Members    ✓     ✓     ✓      ✗    ✗      ✗
Create Task       ✓     ✓     ✓      ✓    ✗      ✗
Delete Task       ✓     ✓     ✓      ✗    ✗      ✗
```

Eine Permission-Regression in Production ist deutlich gefährlicher als ein UI-Bug.

14. Production Pagination

Keine unbegrenzte Collection ohne passende Limits exposen.

Der Client sollte:

```
query GetTasks(
  $first: Int!
  $after: String
) {
  tasks(
    first: $first
    after: $after
```

```
  ) {
    nodes {
      id
      title
    }

    pageInfo {
      hasNextPage
      endCursor
    }
  }
}
```

Der Server muss eine passende max page size haben.

15. N+1 Production Check

Query:

```
projects {
  nodes {
    owner {
      email
    }

    tasks {
      nodes {
        assignee {
          email
        }
      }
    }
  }
}
```

Prüfen: Project.owner, Project.tasks, Task.assignee, Project.members, damit Dataloader/Batching funktioniert.

Eine Query, die korrekte Daten liefert, aber 500 SQL-Queries erzeugt, ist trotzdem ein Production-Bug.

16. Query Depth / Complexity

Das Production-Schema muss die Limits aus Schritt 39 behalten.

Konzept:

```
GraphQL request
      ↓
depth validation
      ↓
complexity validation
      ↓
execution
```

Limits nicht entfernen, nur weil das aktuelle Frontend „nicht tief queried“. Der Client kann trotzdem ungewöhnliche Requests senden.

17. Rate Limiting

Schritt 40: config/initializers/rack_attack.rb

Konzept:

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

100/minute ist nur ein Startbeispiel. Production muss nach realem Traffic angepasst werden.

18. Shared Cache für Rate Limit

Wenn Production Rails A, Rails B und Rails C hat, sollte nicht jede Instanz ihren eigenen Counter halten.

Ein Shared Cache ist nötig, zum Beispiel ein Redis-backed Cache passend zur Infrastruktur.

19. Trusted Proxy

Wenn Browser → Cloudflare → Nginx → Rails, muss geprüft werden, ob request.ip wirklich die Client-IP ist.

Sonst können alle Requests dieselbe Proxy-IP haben und Rack::Attack alle Benutzer gemeinsam drosseln.

20. CORS

Wenn das Frontend https://app.example.com und das Backend https://api.example.com ist, sollte Production CORS nur den benötigten Origin erlauben.

Konzept: origins ENV.fetch("FRONTEND_URL").

Nicht standardmäßig origins "*", besonders bei Credentials/Cookies.

21. CSRF und Devise

Da Devise verwendet wird, muss klar sein: Devise Session Cookie oder Bearer/JWT/Token.

Bei Session Cookie: Browser → Cookie → Rails, dann ist CSRF Protection sehr wichtig.

Nicht einfach aus einem API-Tutorial skip_forgery_protection übernehmen, ohne die Authentication-Architektur zu verstehen.

22. GraphiQL in Production

Development: /graphiql → OK.

Production: /graphiql → normalerweise nicht öffentlich.

Routes können conditional sein:

```
if Rails.env.development?
  mount GraphiQL::Rails::Engine,
        at: "/graphiql",
        graphql_path: "/graphql"
end
```

23. OperationStore / Trusted Documents

Wenn Schritt 42 umgesetzt wurde: Production Client → operationId → OperationStore → trusted document.

Deploy-Reihenfolge: Backend → Sync operations → Frontend.

Nicht das Frontend mit einer neuen operationId deployen, bevor der Server sie kennt.

24. Logging

Der RequestLogger aus Schritt 41 sollte etwa diese Ausgabe liefern:

```
{
  "event": "graphql.execute",
  "operation": "GetTasks",
  "request_id": "abc123",
  "user_id": 15,
  "duration_ms": 84.32,
  "errors": 0,
  "error_codes": [],
  "slow": false
}
```

Bei Schritt 42 kann zusätzlich operation_id ergänzt werden.

25. Keine sensiblen Daten loggen

Das gesamte Projekt prüfen:

```
grep -R "Rails.logger" app/
```

Nach Dingen wie Rails.logger.info(params.inspect), Authorization Header oder variables suchen.

Production sollte password, access token, refresh token, Authorization und sensitive variables nicht loggen.

26. Monitoring

Nach dem Deployment braucht man: requests/minute, errors/minute, p95 latency, p99 latency, slow operations, rate limited requests, DB latency, unexpected exceptions.

Beispiel: GetMe p95 = 40 ms; GetTasks p95 = 180 ms; GetDashboard p95 = 1.8 s → GetDashboard muss untersucht werden.

27. Health Endpoint

Der Health Check sollte keine Dashboard-Query sein, weil sie zu viele Dependencies hat.

Einen separaten Health Endpoint verwenden. Die aktuelle Rails-Version kann /up haben; wenn das Projekt diese Route hat, prüfen:

```
curl https://api.example.com/up

# Kỳ vọng:
HTTP 200
```

28. RSpec vor Deployment

Alles ausführen:

```
bundle exec rspec
```

Nicht nur bundle exec rspec spec/graphql, weil GraphQL von Models, Policies, Services und Database behavior abhängt.

29. Security Tests separat ausführen

Kann so organisiert werden:

```
spec/graphql/security/
├── authorization_spec.rb
├── query_complexity_spec.rb
├── query_depth_spec.rb
└── rate_limit_spec.rb
```

Danach führt CI aus:

```
bundle exec rspec   spec/graphql/security
```

Wenn Security Specs fehlschlagen: STOP DEPLOY.

30. CI Pipeline

Basis-Pipeline: git push → CI → bundle install → database setup → RSpec → security tests → build → deploy.

Konzeptionelles GitHub-Actions-Beispiel:

```
steps:
  - uses: actions/checkout@v4

  - name: Install dependencies
    run: bundle install

  - name: Prepare database
    run: bin/rails db:test:prepare

  - name: Run specs
    run: bundle exec rspec
```

Das konkrete Deployment hängt von der Plattform ab; ohne zu wissen, ob Fly/Render/Heroku/AWS verwendet wird, sollten keine spezifischen Befehle ergänzt werden.

31. Staging

Vor Production: development → test → staging → production.

Staging sollte Production möglichst ähnlich sein: PostgreSQL, Redis, Rack::Attack, GraphQL complexity, GraphQL depth, CORS, authentication.

Nicht Staging mit SQLite + ohne Redis + ohne Rate Limit, wenn Production komplett anders läuft.

32. Smoke Test nach Deployment

Nach dem Backend-Deployment eine einfache Query ausführen:

```
query GetMe {
  me {
    user {
      id
      email
      role
    }
  }
}
```

Danach:

```
query GetProjects(
  $first: Int!
) {
  projects(first: $first) {
    nodes {
      id
      name
    }
  }
}
```

Variables:

```
{
  "first": 5
}
```

Prüfen: authentication ✓, GraphQL ✓, database ✓, authorization ✓, pagination ✓.

33. Smoke Test Mutation

Keine destruktive Mutation auf wichtigen echten Daten testen.

Einen eigenen Test-Project/User für Smoke Tests verwenden.

Beispiel:

```
mutation CreateTask(
  $input: CreateTaskInput!
) {
  createTask(input: $input) {
    task {
      id
      title
    }

    errors
  }
}
```

Nach dem Test: create → verify → cleanup.

34. Direkt nach Deployment monitoren

Direkt nach dem Deployment beobachten: error rate, latency, DB errors, FORBIDDEN spike, VALIDATION_ERROR spike, 429 spike, 500 spike.

Beispiel: vor Deployment VALIDATION_ERROR 5/hour, danach 500/hour → wahrscheinlich wurde der Frontend/Backend-Contract gebrochen.

35. Rollback Plan

Vor dem Deployment muss beantwortet sein: Wie wird Code zurückgerollt? Wie das Frontend? Ist die DB-Migration backward compatible? Laufen alte OperationStore-Operations noch?

Nicht erst bei einem Production-Fehler über Rollback nachdenken.

36. Migration Rollback ist nicht immer sicher

Beispiel remove_column :tasks, :legacy_status. Danach kann bin/rails db:rollback verlorene Daten nicht automatisch wiederherstellen.

Destruktive Migrationen müssen deshalb besonders vorsichtig behandelt werden.

Bevorzugt:

```
deprecate
↓
stop reading
↓
stop writing
↓
wait
↓
remove
```

37. Finaler Deployment Flow

Das ist der empfohlene Flow für das aktuelle Projekt:

```
1. Developer
   ↓
feature branch

2. CI
   ↓
RSpec
   ↓
Permission Matrix
   ↓
Security Specs

3. Merge

4. Deploy Staging
   ↓
db:migrate
   ↓
smoke tests

5. Review staging

6. Deploy Production Backend
   ↓
safe migrations

7. Sync Trusted Operations
```

```
   ↓
nếu dùng Bước 42

8. Health Check

9. GraphQL Smoke Tests

10. Deploy Frontend
    ↓
nếu frontend phụ thuộc schema mới

11. Monitor
    ↓
errors
latency
DB
429

12. Rollback nếu cần
```

38. Checkliste vor Deploy

```
CODE
[ ] bundle exec rspec xanh
[ ] Không debug code
[ ] Không secrets trong Git

DATABASE
[ ] Migration reviewed
[ ] Indexes reviewed
[ ] Constraints reviewed
[ ] DB pool phù hợp
[ ] Backup/rollback strategy

GRAPHQL
[ ] Authentication
[ ] ProjectPolicy
[ ] TaskPolicy
[ ] Policy::Scope
[ ] Permission Matrix
[ ] Pagination
[ ] Max page size
[ ] Dataloader
[ ] Complexity
[ ] Depth limit

SECURITY
[ ] Rack::Attack
[ ] CORS
[ ] CSRF strategy
[ ] GraphiQL không public tùy tiện
[ ] Dashboard admin-only
[ ] Không log token/password

OBSERVABILITY
[ ] operation name
[ ] request_id
[ ] user_id
[ ] duration_ms
[ ] errors
[ ] error_codes
[ ] slow query detection

DEPLOYMENT
[ ] Staging passed
[ ] Backend deploy order
[ ] OperationStore sync nếu dùng
[ ] Health check
[ ] Smoke test
[ ] Frontend deploy
[ ] Monitoring
[ ] Rollback plan
```

39. Definition of Done für Schritt 44

Schritt 44 ist nicht abgeschlossen, nur weil die App deployt werden kann.

Er ist abgeschlossen, wenn:

```
App deploy được
        +
Tests xanh
        +
Permission được kiểm chứng
        +
Migration an toàn
        +
Security limits hoạt động
        +
Logs/metrics hoạt động
        +
Có health check
```

```
        +
Có smoke tests
        +
Có rollback plan
```

Zusammenfassung — Code

Das ist der kurze Code-/Checklisten-Teil zum Speichern.

Schema

```
class YourAppSchema < GraphQL::Schema
  query Types::QueryType
  mutation Types::MutationType

  use GraphQL::Dataloader

  # Depth / Complexity:
  # c□u hình theo version graphql-ruby
  # và s□ liệu thực t□ c□a project.

  # N□u dùng GraphQL Pro:
  # use GraphQL::Pro::OperationStore
end
```

Controller context

```
context = {
  current_user: current_user
}

result =
  YourAppSchema.execute(
    query,
    variables: variables,
    context: context,
    operation_name: operation_name
  )
```

Authorization

```
user =
  authenticate_user!

project =
  Project.find(id)

authorize!(
  project,
  :update?
)
```

Collection scope

```
ProjectPolicy::Scope
  .new(
    current_user,
    Project
  )
  .resolve
```

Database constraint

```
add_index :project_members,
          [:project_id, :user_id],
          unique: true
```

Useful task indexes

```
add_index :tasks,
          [:project_id, :status]

add_index :tasks,
          [:assignee_id, :status]
```

Rate limit

```
class Rack::Attack
  throttle(
    "graphql/ip",
    limit: 100,
    period: 1.minute
  ) do |request|
    request.ip if
      request.post? &&
      request.path == "/graphql"
  end
end
```

Logging

```
{
  event: "graphql.execute",
  operation: operation_name,
  request_id: request.request_id,
  user_id: current_user&.id,
  duration_ms: duration_ms.round(2),
  errors: error_count,
  error_codes: error_codes,
  slow: slow?
}
```

Test

```
bundle exec rspec
```

Production migration

```
RAILS_ENV=production bin/rails db:migrate
```

Health check

```
curl https://api.example.com/up
```

Deploy

```
RSpec
  ↓
Staging
  ↓
Migration
  ↓
Backend
  ↓
OperationStore Sync
  ↓
Health Check
  ↓
GraphQL Smoke Test
  ↓
Frontend
  ↓
Monitoring
```

Der wichtigste Satz aus Schritt 44:

```
Production-ready
≠
"Code chạy được"

Production-ready
=
Code
+ Tests
+ Security
+ Database safety
+ Observability
+ Deployment strategy
+ Rollback
```

Nach Schritt 44 ist Schritt 45 der Zeitpunkt, die gesamte Project Management API in die finale Architektur zusammenzuführen, statt weitere einzelne Security-/Deployment-Features hinzuzufügen.